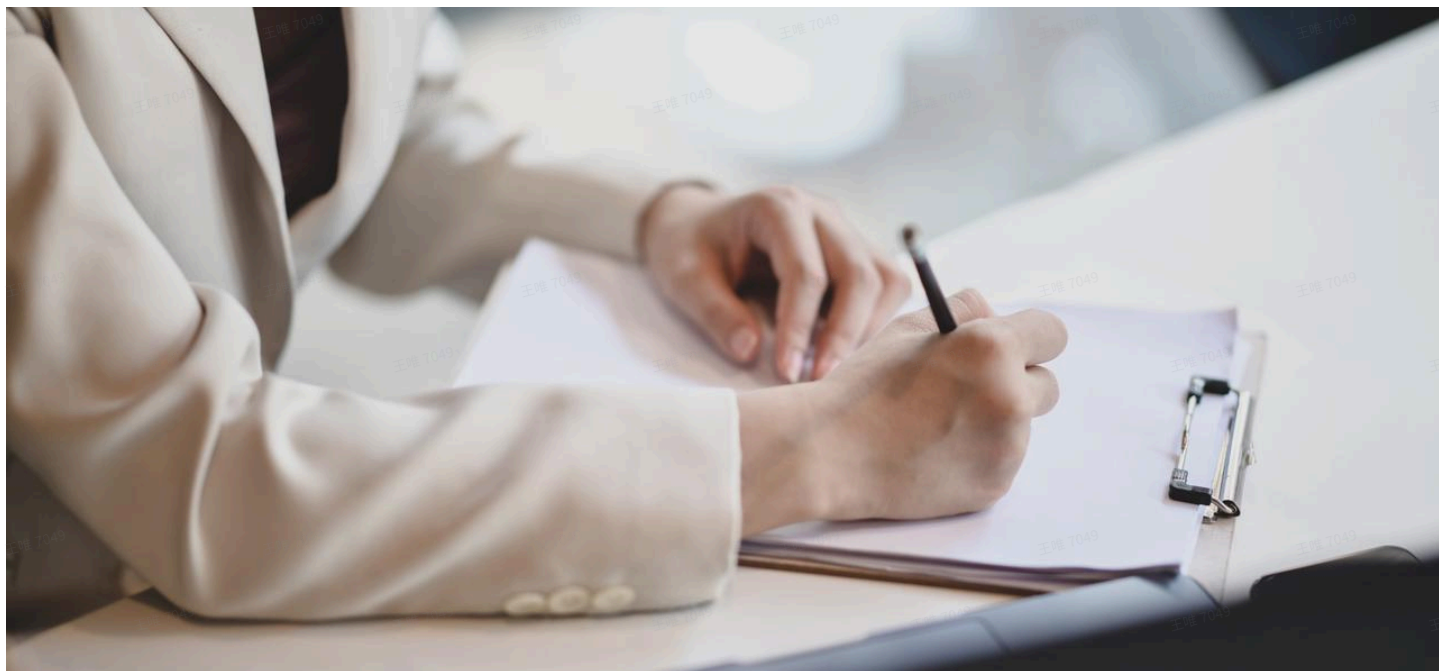




# 【Code With SOLO】用 SOLO 从零搭建外贸报价全链路自动化系统

## 【Code With SOLO】用 SOLO 从零搭建外贸报价全链路自动化系统

[https://www.bilibili.com/video/BV1W8dZB5Ef/?](https://www.bilibili.com/video/BV1W8dZB5Ef/?spm_id_from=333.1387.homepage.video_card.click&vd_source=807ef2d4715bf9b7e13392868c457db9)

[spm\\_id\\_from=333.1387.homepage.video\\_card.click&vd\\_source=807ef2d4715bf9b7e13392868c457db9](spm_id_from=333.1387.homepage.video_card.click&vd_source=807ef2d4715bf9b7e13392868c457db9)

### 一、摘要

使用 TRAE SOLO 从零搭建了一套**外贸报价全链路自动化系统**，实现了从邮箱监听询价邮件 → AI 识别 → 飞书多维表格录入 → 报价单云文档生成 → 用户确认 → PDF 导出 → 自动回复邮件的完整闭环。整个系统基于 Python + lark-cli（飞书命令行工具）构建，包含 9 个模块、15 个 lark-cli 调用点，并通过 SKILL.md 让 AI Agent 也能直接调用飞书工具。原本需要人工 半天到一天 处理的询价报价流程，现在全自动完成，人工仅需在飞书群中确认两次即可。

### 二、背景

我是一名飞书多维表格的独立开发者，此前曾接触到一位外贸公司的老板。了解到外贸公司有几万个产品品类和型号在售，日常工作中最繁琐的环节之一就是**处理客户询价**。每次客户询盘都会咨询几十

上百的品，业务员每天需要耗费4小时查询产品，计算报价，最终产出报价单。之后还需要记录这次报价，同步给业务主管、老板跟进情况。

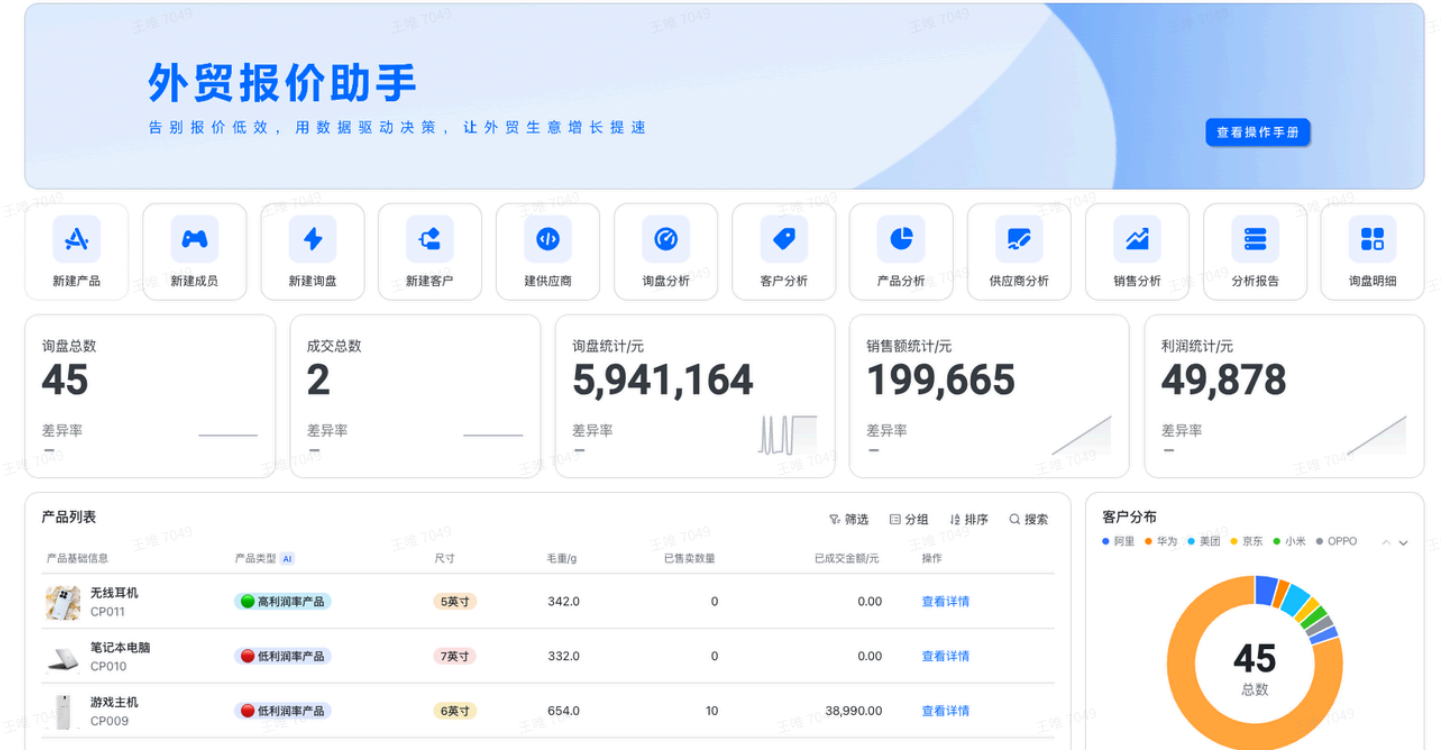
在这里，传统的报价流程如下：

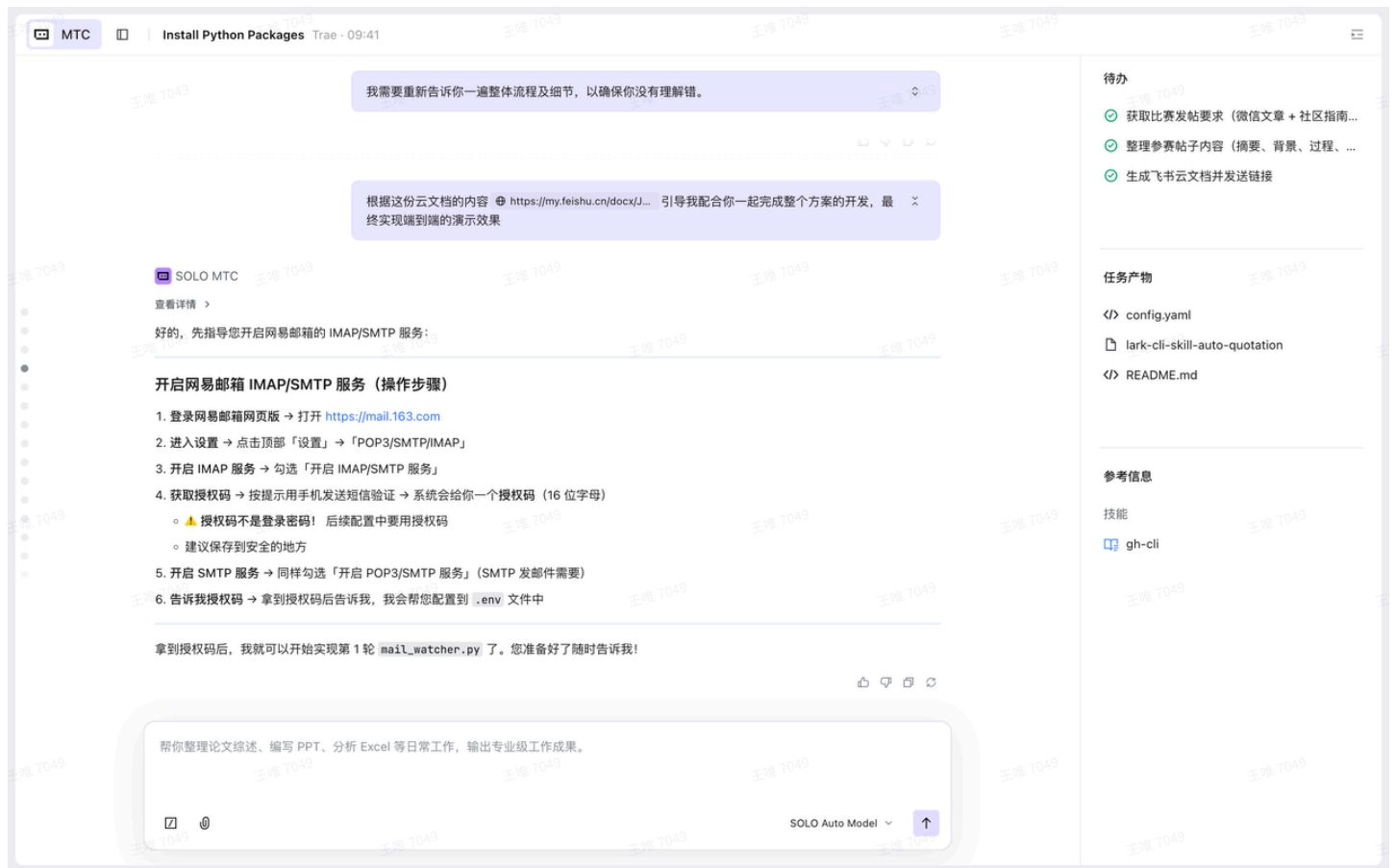
1. 业务员定期检查邮箱，筛选询价邮件（夹杂大量垃圾邮件）
2. 业务员下载询价函 PDF 附件，手动提取产品型号和数量
3. 将询盘需求通过公司系统查询，确认每样产品的价格
4. 汇总计算报价，手动制作报价单（Word/Excel），每一次产出报价单至少需要4小时的时间
5. 业务员和业务主管/老板确认报价单的内容
6. 发送报价邮件给客户，附上 PDF 报价单
7. 和业务主管/老板同步业务情况，能走到这一步，至少1-2天的时间就过去了。

**痛点：**

- 每封询价从收到到回复，人工操作至少需要半天，最多的时候可能需要1天，报价效率极低
- 容易遗漏邮件，尤其是节假日积压时，如果是国内外还会有时差的问题
- 每个业务员制作的报价单格式不统一，人工制作容易出错，影响客户信任
- 重复性劳动，占用了业务员大量本可用于跟进客户的时间
- 人工报价后，记录多依赖业务员手动整理，易出现记录遗漏、信息不全的情况，无法形成完整的报价档案
- 业务主管和老板难以实时获取所有报价情况，无法及时跟进客户反馈、分析报价转化率，不利于业务复盘和管理决策

**我的目标：** 结合我此前荣获2025年度十佳应用的外贸报价助手（具体方案见：[国外贸报价助手，让询盘报价的过程从4小时缩短为1分钟](#)），用 SOLO 搭建一个端到端的自动化系统，把人工操作压缩到最少1分钟不到——业务员只在需要决策的环节（确认报价、确认邮件内容）介入即可，业务情况的跟进、复盘则通过飞书多维表格搭建的外贸报价助手实现。





## 邮件监听（mail\_watcher.py）：

- 基于 Python imaplib 实现 IMAP IDLE 监听
- SOLO 帮我处理了断线重连、超时控制等边界情况

## AI 询价识别（inquiry\_detector.py）：

- 使用火山方舟豆包大模型（OpenAI 兼容接口）
- SOLO 设计了「关键词预筛 + AI 二次确认」的双重策略，有效降低误判
- 还实现了从 PDF 询价函中自动提取产品需求的 Prompt

## 飞书多维表格读写（base\_writer.py）：

- 通过 lark-cli 命令行工具操作飞书 API
- SOLO 帮我实现了创建记录、上传附件、查询报价、更新状态等完整 CRUD
- 特别处理了附件上传的 0B bug（lark-cli 已知问题），采用 drive + upload → api PATCH 两步流程绕过

## 报价单生成（quotation\_generator.py）：

- 生成飞书云文档（Markdown 格式），供用户在线编辑
- 从用户确认后的云文档直接生成 PDF（WeasyPrint）
- SOLO 帮我解决了飞书 Markdown 中 lark-table 标签的解析问题

## 邮件发送（mail\_sender.py）：

- SMTP SSL 发送，支持邮件线程（In-Reply-To / References）
- SOLO 加入了 3 次重试机制和超时保护

### 第三步：用户确认机制

这是整个系统最关键的交互设计。我没有使用终端 input()（因为系统是后台运行的守护进程），而是 SOLO 帮我设计了飞书群消息轮询确认机制：

- 系统在飞书群发送通知卡片（含云文档链接）
- 用户在云文档中调整报价内容
- 用户在群内回复「确认」或「取消」
- 系统每 10 秒轮询群消息，检测用户回复

整个流程有 2 个确认节点：

#### 1. 报价单确认 → 确认后生成 PDF

庄工 > 报价单 QT-20260419-904 - 苹果贸易有限公司 平

云盘 最近修改: 2 小时前

分享 编辑

报价单 QT-20260419-904 - 苹果贸易有限公司

庄工 | 今天修改

报价单

产品明细

付款方式

交货期

包装方式

银行信息

报价编号: QT-20260419-904客户名称: 苹果贸易有限公司报价日期: 2026-04-19有效期至: 2026-05-19

产品明细

| 序号 | 产品名称  | 型号规格  | 数量  | 单价 (USD) | 金额 (USD)     |
|----|-------|-------|-----|----------|--------------|
| 1  | 智能手机  | CP001 | 50  | 4,999.00 | 249,950.00   |
| 2  | 笔记本电脑 | CP003 | 30  | 7,399.00 | 221,970.00   |
| 3  | 笔记本电脑 | CP010 | 20  | 3,499.00 | 69,980.00    |
| 4  | 无线耳机  | CP002 | 100 | 6,999.00 | 699,900.00   |
|    |       |       | 合计  |          | 1,240,000.00 |

付款方式

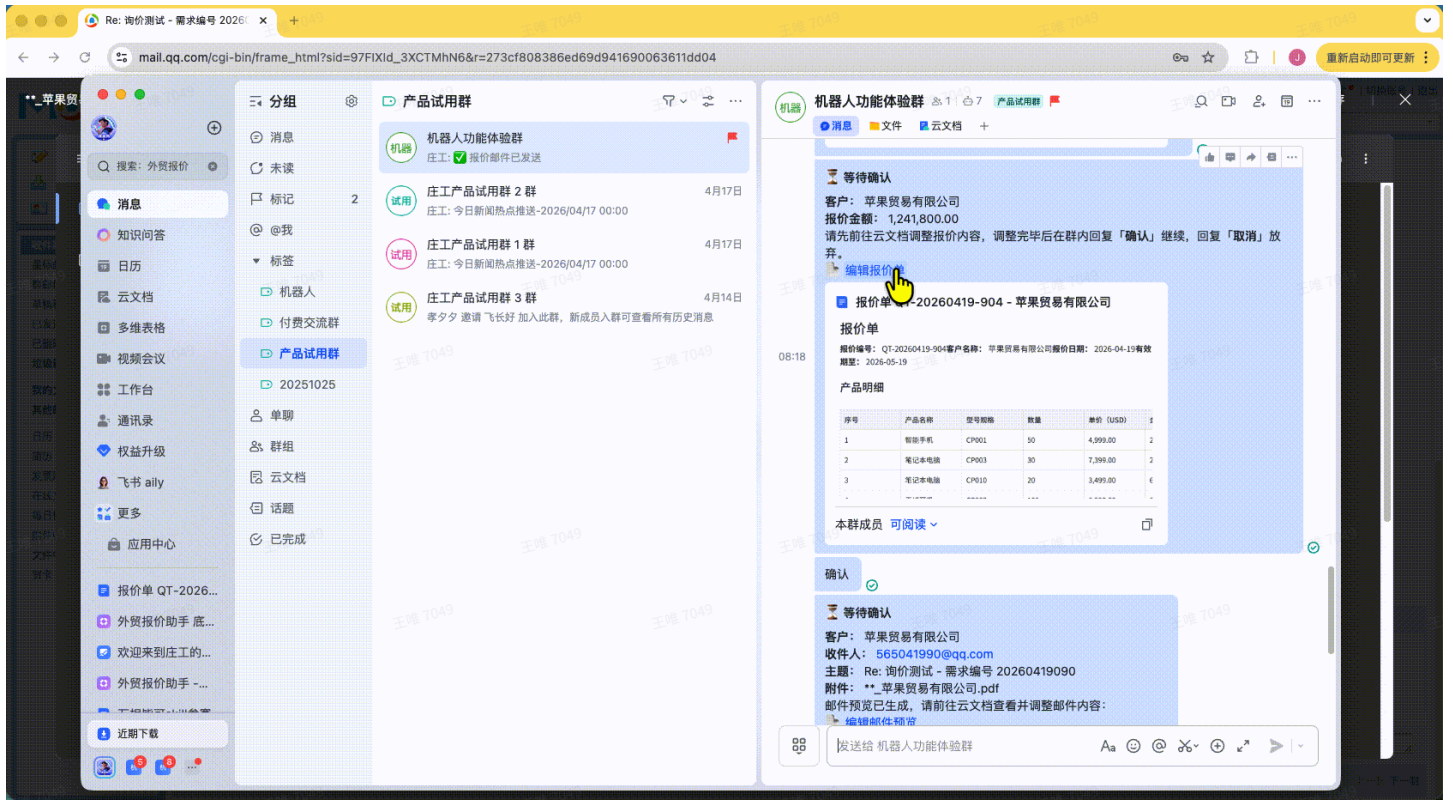
电汇 30% 订金, 发货前付 70% 尾款, 123

交货期

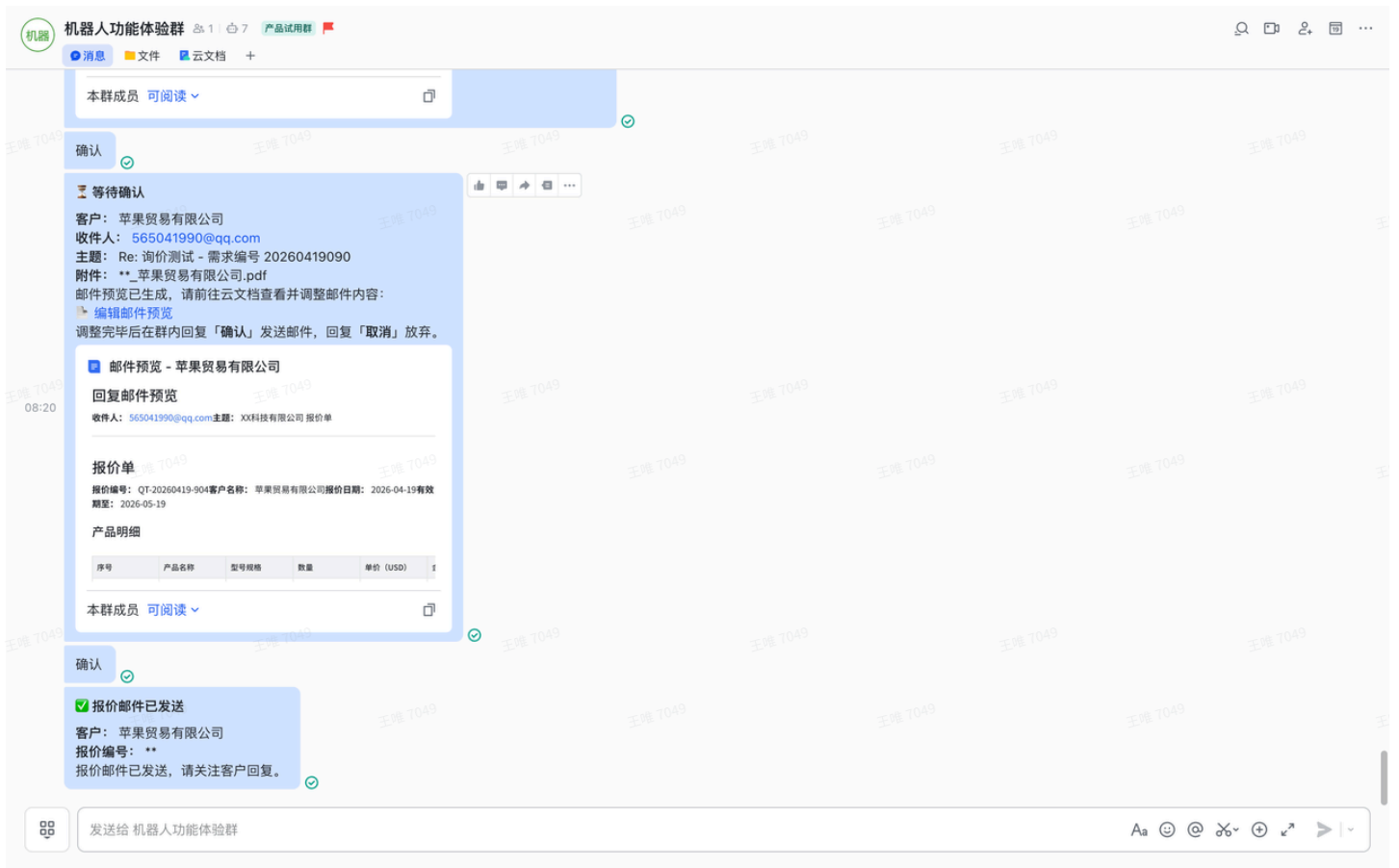
收到订单后 15 个工作日内交货

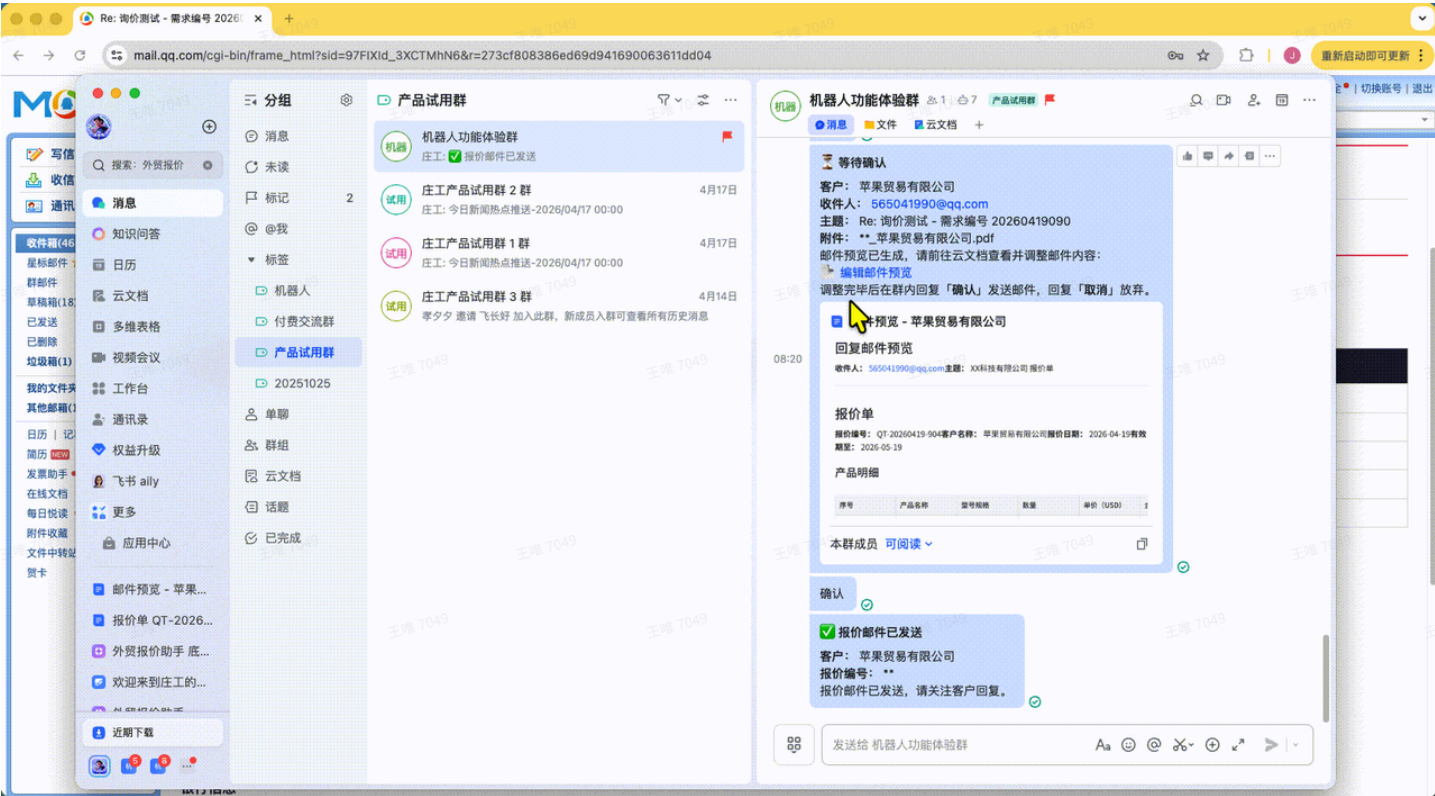
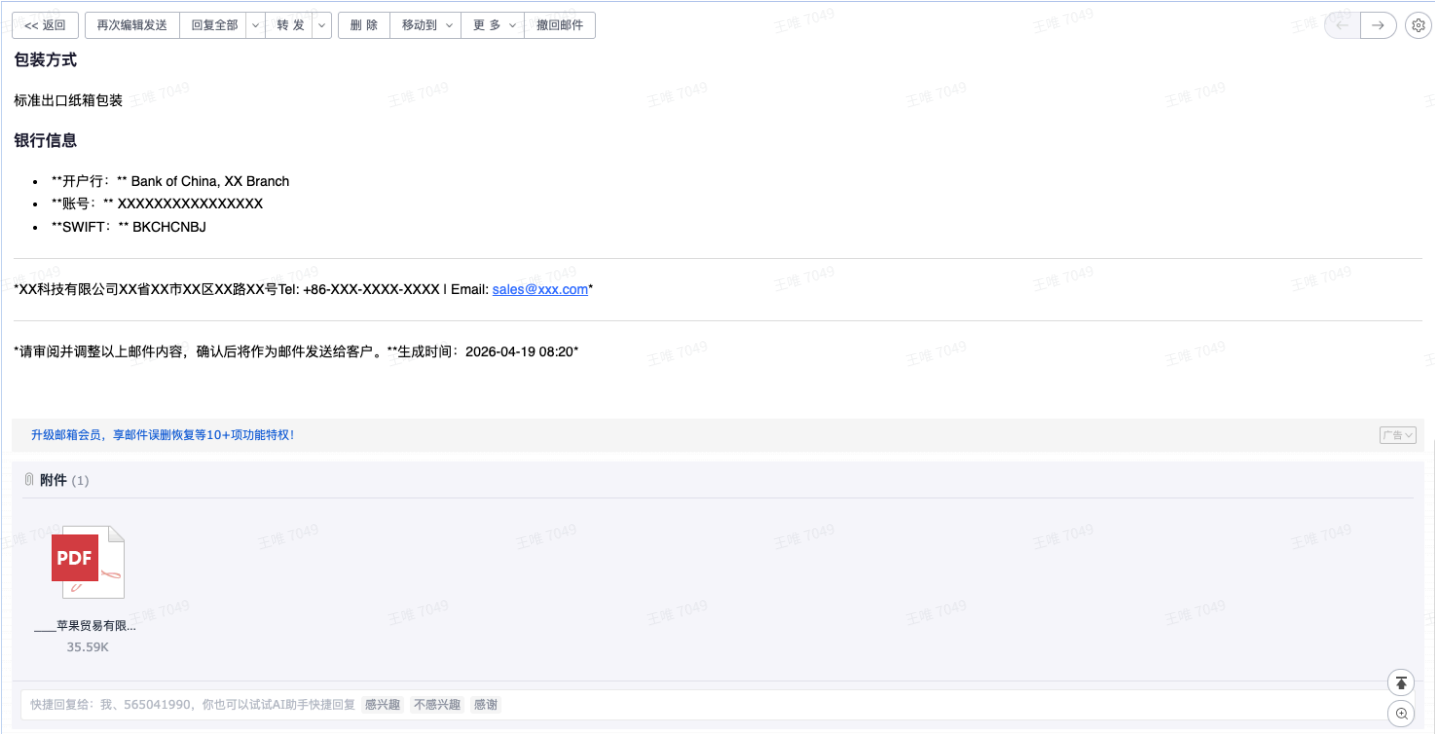
包装方式

标准出口纸箱包装



## 2. 邮件预览确认 → 确认后发送邮件





## 第四步：踩坑与迭代

开发过程中遇到了大量问题，SOLO 帮我逐一解决：

| 问题        | 解决方案                                                       |
|-----------|------------------------------------------------------------|
| 回复邮件收件人取错 | 163/QQ 邮箱的 From 头可伪造，改用 X-Sender > Return-Path > From 优先级链 |
| 报价单产品数量不对 |                                                            |

|                      |                                         |
|----------------------|-----------------------------------------|
|                      | 轮询时未按需求编号过滤，导致拉取了所有产品，加入 req_number 过滤  |
| PDF 内容与云文档不一致        | 原来是重新解析再生成，改为直接将云文档 Markdown 转 HTML/PDF |
| PDF 在用户确认前就生成了       | 重构流程：先生成云文档 → 用户确认 → 再生成 PDF            |
| lark-cli 附件上传 0B bug | 改用 drive +upload → api PATCH 两步流程       |
| 轮询卡死                 | limit 从 100 改为 500，新数据不再被截断             |

## 第五步：编写 SKILL.md

这是项目的亮点之一。SOLO 帮我分析了整个项目中所有的 lark-cli 调用点（共 15 处），将它们分为两类：

### 适合编写为 Skill 指令的（11 条）：

- 无状态、单次执行的 lark-cli 命令（创建记录、查询数据、创建云文档、发送通知等）
- AI Agent 可以直接在终端执行这些命令

### 不适合编写为 Skill 的（7 个模块）：

- 有状态、长连接、复杂依赖的操作（IMAP 监听、AI 识别、PDF 生成、SMTP 发送等）
- 维持 Python 代码实现

最终生成了结构清晰的 SKILL.md，包含完整的命令示例、参数说明、返回值格式和注意事项。

## 第六步：代码审查与加固

SOLO 对全部 9 个 Python 文件进行了全面代码审查，发现并修复了 22 个问题，包括：

- 配置校验（启动时检查必填字段）
- 网络超时（IMAP/SMTP/subprocess 统一设置超时）
- 重试机制（SMTP 3 次、AI 2 次）
- 异常保护（主循环 try/except，单封邮件失败不影响后续处理）
- 资源泄漏（fitz 文档上下文管理器）

## 四、成果展示

### 项目地址

GitHub 仓库：<https://github.com/peachgreenti/lark-cli-skill-auto-quotation>

### 系统架构

|     |                                                       |
|-----|-------------------------------------------------------|
| 代码块 |                                                       |
| 2   | Python 自动化流水线                                         |
| 3   | main.py (编排)                                          |
| 4   | ├─ mail_watcher.py ← IMAP 长连接监听                       |
| 5   | ├─ mail_parser.py ← 邮件解析 (X-Sender/Return-Path)       |
| 6   | ├─ inquiry_detector.py ← AI 询价识别 (OpenAI API)         |
| 7   | ├─ base_writer.py ← 飞书多维表格 CRUD (lark-cli base)       |
| 8   | ├─ quotation_poller.py ← 轮询报价结果 (lark-cli base)       |
| 9   | ├─ quotation_generator.py ← 报价单/PDF生成 (lark-cli docs) |
| 10  | ├─ mail_sender.py ← SMTP 回复邮件                         |
| 11  | └─ notifier.py ← 飞书群通知 (lark-cli im)                  |
| 12  |                                                       |
| 13  |                                                       |

项目文件结构

|     |                                               |
|-----|-----------------------------------------------|
| 代码块 |                                               |
| 1   | lark-cli-skill-auto-quotation/                |
| 2   | ├─ main.py # 入口：流程编排、飞书群消息轮询                  |
| 3   | ├─ mail_watcher.py # IMAP 邮件监听                |
| 4   | ├─ mail_parser.py # 邮件解析 (头、正文、附件)            |
| 5   | ├─ inquiry_detector.py # AI 询价识别 + 需求提取       |
| 6   | ├─ base_writer.py # 飞书多维表格读写                  |
| 7   | ├─ quotation_poller.py # 报价结果轮询               |
| 8   | ├─ quotation_generator.py # 报价单生成 (云文档 + PDF) |
| 9   | ├─ mail_sender.py # SMTP 邮件发送                 |
| 10  | ├─ notifier.py # 飞书群通知                        |
| 11  | ├─ config.yaml # 配置文件                         |
| 12  | ├─ requirements.txt # Python 依赖               |
| 13  | ├─ SKILL.md # AI Agent 调用指令                   |
| 14  | ├─ README.md # 完整项目文档                         |
| 15  | └─ .gitignore                                 |
| 16  |                                               |

技术栈

| 组件   | 技术             |
|------|----------------|
| 语言   | Python 3.10+   |
| 邮件监听 | IMAP (imaplib) |
| 邮件发送 | SMTP (smtplib) |

|        |                          |
|--------|--------------------------|
| PDF 生成 | WeasyPrint (HTML → PDF)  |
| PDF 读取 | PyMuPDF (fitz)           |
| AI 模型  | 火山方舟 豆包大模型 (OpenAI 兼容接口) |
| 飞书集成   | lark-cli (命令行工具)         |
| 配置管理   | YAML + python-dotenv     |

## SKILL.md 亮点

SKILL.md 将项目中 11 条 lark-cli 指令整理为 AI Agent 可直接调用的工具文档，每条指令包含：

- 完整的命令示例和参数说明
- 返回值格式（JSON 结构）
- 关键注意事项（如 doc\_id vs doc\_token、create\_time 格式、content 二次解析）
- 4 个典型操作流程（创建询盘、查询报价、生成文档、发送通知）

## 五、效果与总结

### 方案价值

这套方案最大的价值在于**降本、提效、增收**。

假设1个业务员的月薪是6000元，每天工作8小时，却要花费4小时的时间用于整理报价单，每个月就浪费了3000元，1年就是36000元，10个业务员1年就浪费了36万元！这还不包括定期的邮件检查、领导确认与回复、业务汇报、业务分析和管理协同的时间，以及因报价滞后而导致流失的商机。

而这套方案的实施省掉了这些时间和浪费的成本，业务员可以把更多的时间用于业务沟通、客户开发，极大地提升了业务效率：

- 对业务员来说，不必天天盯着邮箱，不用再浪费时间做报价单，不需要定期整理业务情况，AI 可以配合他汇报并分析业务进展，员工满意度提升，业务员只需要让更多的人知道他能提供什么产品以及他的联系方式；
- 对业务主管、老板来说，不用每天追在业务员屁股后面追问业务情况，让业务员耗费时间分析成败原因，浪费过多的时间在无效的内部会议上；
- 对客户来说，不需要每次询盘等待几个小时才能知道报价，几分钟就得到报价的体验会让客户印象分大增！

毋庸置疑，更快的报价速度意味着更高的成单几率！

**这是一套三赢的解决方案！**

### 提效数据

| 指标       | 改造前       | 改造后          |
|----------|-----------|--------------|
| 单封询价处理时间 | 0.5-1天    | 约 1分钟（仅确认操作） |
| 邮件遗漏率    | 偶尔遗漏      | 0%（自动监听）     |
| 报价单格式    | 人工制作，不统一  | 模板自动生成，格式统一  |
| 人工介入环节   | 全流程 10+ 步 | 仅 2 次确认      |

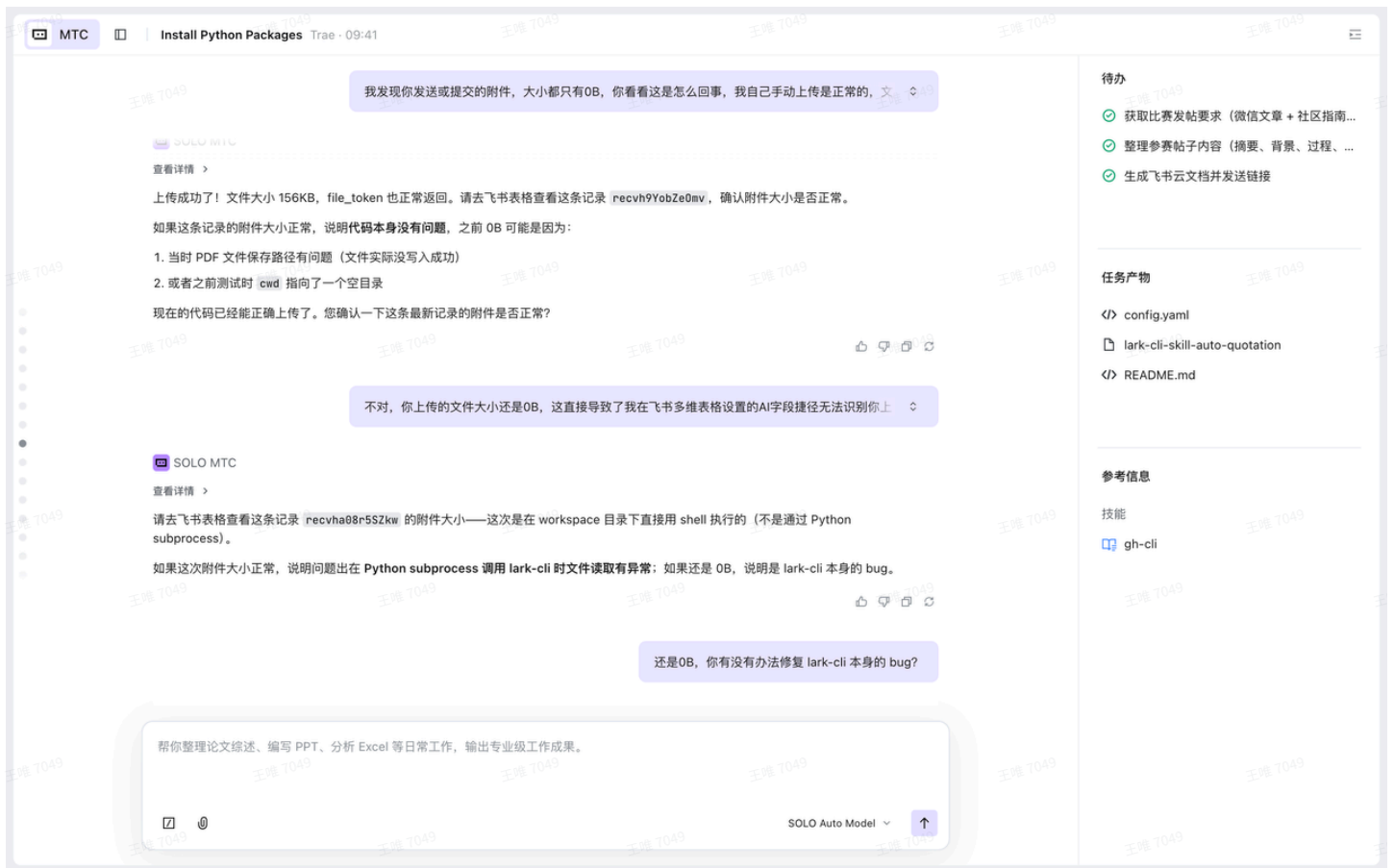
## SOLO 在整个过程中的作用

SOLO 不仅仅是一个代码生成工具，它在整个项目中扮演了**技术合伙人**的角色：

- 1. **需求分析阶段**：帮我梳理业务流程，拆解出 10 个阶段的流水线架构
- 1. **编码实现阶段**：逐个实现 9 个模块，处理了大量边界情况（邮件头解析、附件上传 bug、PDF 格式转换等）
- 2. **调试迭代阶段**：在端到端测试中，快速定位并修复了收件人取错、数据过滤、PDF 不一致等多个问题
- 3. **代码审查阶段**：全面审查 9 个文件，发现并修复 22 个潜在问题
- 4. **文档编写阶段**：生成了完整的 README.md 和 SKILL.md

## 使用心得

- 1. **遗憾**：因为庄工是飞书个人号，没有飞书的邮箱功能，不然就可以直接调用飞书CLI去抓到飞书邮箱的收件箱，第一步可能会更快；飞书CLI或是多维表格可能自带Bug，导致通过飞书CLI上传到飞书多维表格的附件大小永远是0B，只能让SOLO帮我解析附件内容。



2. **拆解任务比直接写代码更重要。**我花了不少时间让 SOLO 理解业务流程，这个投入非常值得——后续编码几乎没走弯路。
3. **迭代式开发是关键。**不要期望一次生成完美代码，而是先跑通主流程，再逐步修复边界情况。SOLO 在调试阶段的价值远大于初次生成。
4. **把确认环节交给飞书群是正确的决定。**系统是后台守护进程，不能用终端 `input()`，SOLO 帮我设计的群消息轮询方案既实用又优雅。
5. **SKILL.md 是意外收获。**原本只是为了整理文档，结果发现它让 AI Agent 也能直接操作飞书，实现了「人机协作」的另一种模式。
6. **真实场景 > 复杂技术。**这个项目用的都是成熟的技术（IMAP、SMTP、lark-cli），叠加飞书多维表格（需要有一套稳定的数据库记录业务数据，方便业务调用），没有花哨的框架，但解决的是真实的工作痛点，这比技术炫技更有价值。
7. **未来：多渠道监听 → 智能体派单 → OPC。**从监听邮件、飞书逐步到企微、钉钉，甚至是QQ/微信的消息，业务员只需要每天通过各种渠道让别人知道自己和联系方式即可；这一步之上，有了大量数据的积累，就可以开发报价智能体，用同一个邮箱接收所有的询盘，智能体根据业务员的繁忙情况派单，由业务员人工确认跟进。业务员的跟进职能进一步弱化，未来每个人都有可能基于此直接做个OPC。

本项目使用 TRAE SOLO 从零搭建，全程通过自然语言对话完成开发、调试、审查和文档编写。